Universidad San Carlos de Guatemala
Facultad de ingeniería.
Ingeniería en ciencias y sistemas

Título del Proyecto:

Sistema Integral de Monitoreo y Seguridad del Sistema Operativo

PONDERACIÓN: 60 pts

Tiempo estimado: 20 hrs

Índice

1. MARCO FORMATIVO.... 4
1.1. Valor.... 4
1.2. Competencia(s).... 4
1.3. Objetivo SMART.... 4
2. Enunciado del Proyecto.... 5
2.1 Descripción del problema a resolver.... 5
2.2 Alcance del proyecto.... 6
2.2.1 Syscalls previamente implementadas.... 6
2.2.2 Monitoreo de archivos.... 7
1. Configurar la ruta a analizar.... 7
2. Actualizar la blacklist y analizar eventos.... 7
2.2.3 Espacio de Usuario (Daemon en C).... 8
1. Monitoreo del sistema.... 8
2. Listado de procesos.... 9
3. Listado de archivos analizados.... 9
4. Archivos en cuarentena.... 9
8. Terminación de Procesos.... 9
9. Sistema de alertas.... 9
10. Gestión de usuarios (PAM + Login).... 10
11. Integración con log_watch (canal IPC).... 12
2.2.4 Dashboard Web (Frontend).... 12
1. Visualización de métricas.... 13
2. Panel de alertas.... 13
3. Visualización de archivos analizados.... 13
4. Visualización de amenazas detectadas.... 14
5. Visualización de procesos activos.... 14
6. Análisis de palabras clave.... 14
7. Sistema de autenticación.... 15
2.3 Requerimientos técnicos.... 16
3. Entregables.... 17
4. Material de apoyo.... 17
5. Recursos y herramientas a utilizar.... 18
6. Cronograma.... 18
7. Rúbrica de Calificación.... 19
7.1 Requisitos para optar a la calificación.... 19
7.2 Resumen de Puntuaciones.... 20

1. MARCO FORMATIVO

1.1. Valor

Nombre del valor	¿Cómo se aplica en tu laboratorio?
Adaptabilidad	Explorar diferentes enfoques es indispensable para aumentar nuestra biblioteca personal de conocimiento. Una solución puede ser implementada de muchas formas, elegir la forma correcta es crítico para el futuro del sistema y por eso debe analizarse con cuidado cómo adaptarse a nuevos entornos.

1.2. Competencia(s)

Tipo de Competencia	
Competencia General	Aplica principios básicos de ingeniería, ciencias de computación y sistemas de información y comunicación, en la formulación y resolución adecuada de problemas complejos.
Competencia Específica	Demuestra pensamiento crítico, actitud investigativa y rigor analítico en el planteamiento y la resolución de problemas complejos.

1.3. Objetivo SMART

Específico (¿Qué?)	**Medible (¿Cuánto?)**	**Alcanzable (¿Cómo?)**	**Realista (¿Para qué?)**	**A Tiempo (¿Cuándo?)**
Diseñar e implementar un sistema integral de monitoreo y análisis de seguridad a nivel kernel, que permita recolectar métricas del sistema, analizar procesos y archivos mediante llamadas al sistema personalizadas, y detectar amenazas simuladas	El sistema se considerará completo cuando se implementan al menos 5 métricas del sistema, un mecanismo funcional de detección basado en una blacklist de amenazas simuladas, un sistema de alertas con clasificación de severidad (LOW, MEDIUM,	Se logrará mediante la integración en el kernel de Linux de las syscalls desarrolladas en las prácticas anteriores (información de procesos, canal IPC y monitoreo de logs), la implementación de un sensor eBPF para interceptar eventos del sistema de archivos, el desarrollo de un	Para comprender el funcionamiento interno de los sistemas operativos, el flujo de comunicación entre kernel y espacio de usuario, y la aplicación de conceptos de ciberseguridad como detección basada en firmas, monitoreo de recursos y control de acceso,	El proyecto deberá desarrollarse dentro del período establecido para el proyecto, cumpliendo con las etapas de implementación del kernel, desarrollo del programa intermedio, integración del sistema y visualización final en el dashboard.

utilizando un mecanismo de firmas basado en hashes.	HIGH), y un dashboard que visualice correctamente la información en tiempo real.	programa en espacio de usuario en C (daemon) que procese la información, aplique la lógica de detección y la envíe a un dashboard web, y la construcción de dicho dashboard con autenticación mediante PAM.	simulando el comportamiento de herramientas reales de seguridad informática.	

2. Enunciado del Proyecto

2.1 Descripción del problema a resolver

En los sistemas operativos modernos, la capacidad de monitorear y analizar el comportamiento del sistema en tiempo real es fundamental para garantizar su estabilidad, rendimiento y seguridad. Herramientas tradicionales como *top* o *htop* permiten observar el consumo de recursos, pero no incorporan mecanismos avanzados de detección de amenazas ni análisis de comportamiento a nivel kernel.

Por otro lado, las soluciones antivirus convencionales operan principalmente en espacio de usuario, lo que limita su capacidad de acceso directo a estructuras internas del sistema operativo y reduce su efectividad para detectar comportamientos anómalos en tiempo real.

Además, muchos sistemas carecen de mecanismos integrados que permitan correlacionar métricas del sistema con eventos de seguridad, como modificaciones sospechosas de archivos o ejecución de procesos con patrones inusuales.

Ante esta problemática, se plantea la necesidad de desarrollar un sistema integral que combine capacidades de monitoreo de recursos, análisis de procesos y detección de amenazas, utilizando una arquitectura que permita la interacción directa con el kernel mediante llamadas al sistema personalizadas (syscalls).

El problema a resolver consiste en diseñar e implementar un sistema capaz de:

- Recolectar métricas del sistema en tiempo real desde el kernel, incluyendo uso de memoria, estado de páginas y comportamiento de procesos.
- Detectar archivos potencialmente maliciosos mediante un mecanismo de firmas basado en hashes (hash blacklist), simulando el funcionamiento de un motor antivirus.

- Identificar cambios en archivos del sistema mediante comparación de hashes y timestamps, permitiendo detectar modificaciones no esperadas.
- Generar alertas de seguridad clasificadas por nivel de severidad (LOW, MEDIUM, HIGH), en función de las condiciones detectadas.
- Controlar el acceso y las funcionalidades del sistema mediante autenticación de usuarios utilizando PAM, diferenciando entre usuarios con privilegios administrativos y usuarios estándar.

Para lograrlo, el sistema deberá integrar múltiples componentes: un conjunto de syscalls implementadas en el kernel para la obtención de información, un programa intermedio en espacio de usuario encargado de procesar y transmitir los datos y que gestione la lógica de análisis y detección de amenazas, y un dashboard web que permita la visualización de métricas y alertas en tiempo real.

El objetivo es que el estudiante comprenda de manera integral cómo se construyen herramientas de observabilidad y seguridad desde el nivel más bajo del sistema operativo, y cómo estas pueden integrarse en una arquitectura completa que simule el comportamiento de soluciones reales de monitoreo y ciberseguridad.

2.2 Alcance del proyecto

El sistema a desarrollar consiste en una **plataforma integral de monitoreo y análisis de seguridad a nivel kernel**, compuesta por múltiples módulos interconectados que permiten recolectar, procesar, analizar y visualizar información del sistema en tiempo real.

El sistema se divide en tres capas principales que interactúan entre sí:

2.2.1 Syscalls previamente implementadas

El proyecto no requiere implementar syscalls nuevas. Se deben reutilizar e integrar las desarrolladas en las prácticas anteriores, y el estudiante puede tomar como base el código del daemon y del dashboard de la Práctica 4, adaptándolo a los nuevos requerimientos:

- `sys_get_process_info(pid)`: Obtiene información detallada de un proceso específico (implementada en la Práctica 2).
- `sys_get_processes_state()`: Retorna el estado de los procesos activos del sistema (implementada en la Práctica 2 e integrada en la Práctica 4). Es la base para obtener el top 20 de procesos por consumo de memoria. Debe incluir, por proceso, al menos PID, nombre, estado y memoria utilizada (RSS). Si la implementación del estudiante no incluye la memoria, puede complementarla leyendo /proc/

Restricciones importantes Syscall (Kernel)

- Todas las syscalls deben implementarse en C.
- Deben registrarse correctamente en el kernel.
- No se permite duplicar funcionalidad entre syscalls.

- No debe afectar la estabilidad del sistema operativo.

2.2.2 Monitoreo de archivos

A su vez, se debe implementar un programa eBPF que intercepte eventos del sistema de archivos (creación/modificación) utilizando tracepoints o kprobes (ej. sys_enter_openat, sys_enter_write) en un directorio específico. El programa eBPF deberá enviar los metadatos capturados (ruta, acción) al espacio de usuario de forma asíncrona y eficiente utilizando un eBPF Ring Buffer o Perf Buffer. El programa eBPF debe ignorar los eventos generados por el propio daemon (por ejemplo, comparando el PID del proceso que ejecuta la syscall contra el PID del daemon, almacenado en un mapa eBPF). De lo contrario, al calcular el SHA-256 el daemon abriría el archivo, generaría un nuevo evento y se produciría un bucle infinito.

1. Configurar la ruta a analizar

El proyecto requiere detectar archivos mediante firmas basadas en hashes (SHA-256) utilizando un archivo `blacklist.json`. El flujo consta de:

1. **En el Daemon (Espacio de usuario):** El administrador configura la ruta desde el Dashboard. El Daemon en C recibe esta ruta, y utiliza una función de `libbpf` (como `bpf_map_update_elem()`) para insertar o actualizar la ruta en el mapa de eBPF.
2. **En el Sensor (Espacio de kernel):** Cada vez que ocurre un evento de archivo (ej. `sys_enter_openat`), el programa eBPF se ejecuta. Lo primero que hace es leer el mapa de configuración usando `bpf_map_lookup_elem()`.
3. **Filtrado en el Kernel:** El programa eBPF compara la ruta del archivo que se está abriendo o escribiendo con la ruta almacenada en el mapa. Hay coincidencia únicamente cuando el archivo está directamente dentro del directorio configurado (no se incluyen subdirectorios). Si coincide y el proceso no es el propio daemon, envía el evento mediante el Ring Buffer. En caso contrario, lo descarta. Este filtro por ruta es la única lógica permitida en el programa eBPF.

2. Actualizar la blacklist y analizar eventos

1. **Actualización de la Blacklist (Daemon):** Cuando el administrador sube un nuevo archivo JSON desde el Dashboard, el Daemon en C lee el archivo, parsea el JSON y carga todos los hashes maliciosos en la memoria del programa de usuario (por ejemplo, en una tabla hash en C).
2. **Intercepción (eBPF):** El programa eBPF se encarga de vigilar la ruta configurada. Cuando detecta que un archivo en esa ruta fue creado o modificado y cerrado, envía un mensaje rápido al Ring Buffer indicando: *"El archivo /ruta/monitoreada/archivo.txt fue modificado por el PID 1234"*.
3. **Análisis y Alerta (Daemon):** El hilo de tu Daemon en C que está escuchando el Ring Buffer recibe la notificación. El Daemon va al disco, **calcula el SHA-256** del

archivo modificado, y lo compara con la lista de hashes (Blacklist) que parseó del JSON.
4. **Respuesta (Daemon):** Si el hash coincide con uno malicioso en la blacklist, el Daemon genera la alerta de nivel **HIGH** y procede a poner el archivo en cuarentena (simulado) cambiando los permisos a `0000`.
5. El programa eBPF únicamente filtra por ruta y envía metadatos (ruta, acción, PID) al espacio de usuario mediante Ring Buffer/Perf Buffer. El cálculo de SHA-256, la clasificación de alertas y las decisiones de seguridad deben realizarse en el daemon en C.
6. Cuando el sensor detecta una modificación de archivo, el daemon recalcula su hash SHA-256 y compara el timestamp. Si el archivo no coincide con la blacklist pero su hash o timestamp cambió, se genera una alerta **LOW** indicando la alteración.

Si el SHA-256 calculado coincide con un hash de la blacklist, el daemon clasifica el evento con severidad HIGH, genera la alerta correspondiente y aplica cuarentena automática cambiando los permisos del archivo a 0000.

2.2.3 Espacio de Usuario (Daemon en C)

Este componente será el núcleo del sistema en espacio de usuario.

Funciones principales

El programa deberá:

- Recopilar métricas del sistema
- Analizar procesos
- Escanear archivos del sistema
- Generar estructuras de datos en formato JSON
- Enviar los datos al dashboard (HTTP/SE o WebSocket)
- Autentica a los usuarios usando PAM, definiendo roles de *admin_user* y *common_user*.

Ejecución

- Debe ejecutarse como un **daemon** (segundo plano)
- Debe permitir ejecución continua
- Debe manejar errores sin finalizar abruptamente

1. Monitoreo del sistema

El daemon deberá leer periódicamente /proc/meminfo para obtener métricas de memoria (MemTotal, MemFree, MemAvailable, Buffers, Cached, **Active, Inactive**, SwapTotal, SwapFree) y **/proc/vmstat** para registrar los fallos de página menores (pgfault) y mayores (pgmajfault), calculando sus deltas entre intervalos de muestreo.

2. Listado de procesos

Se deberá obtener periódicamente el listado de procesos activos en el sistema utilizando la llamada al sistema ***sys_get_processes_state***, de los cuales se debe *filtrar el top 20 (desde espacio de usuario)* procesos con mayor consumo de memoria. A su vez se tendrá la opción de obtener la información de un proceso utilizando la llamada al sistema ***sys_get_process_info***

3. Listado de archivos analizados

Se debe de mostrar el listado de archivos analizados por el sensor eBPF, mostrando si el archivo es malicioso (fue identificado en el blacklist) o benigno.

4. Archivos en cuarentena

Cuarentena automática: cuando se detecta un hash malicioso, el daemon cambia los permisos del archivo a 0000 (chmod 0000). Cuarentena manual: el administrador puede poner un archivo en cuarentena desde el dashboard, con el mismo efecto. El archivo no podrá ser leído, escrito ni ejecutado.

7. Terminación de Procesos

Opción que permite matar un proceso utilizando la llamada al sistema ***sys_kill*** ya existente en el kernel de Linux. No se permite terminar el proceso con PID 1, el propio daemon ni hilos del kernel.

8. Sistema de alertas

El daemon debe analizar continuamente las métricas recibidas y los eventos del sensor eBPF para clasificar y generar alertas.

Las alertas se deberán de generar y reenviar al frontend, indicando el **motivo**, una **descripción** y la **severidad**.

Las alertas deberán clasificarse en LOW, MEDIUM y HIGH según la siguiente lista:

- LOW
  - Archivo modificado
  - Monitoreo del sistema activado
  - Inicio de sesión exitoso
- MEDIUM
  - Monitoreo del sistema desactivado
  - Inicio de sesión fallido
  - Palabra clave detectada en log (log_watch)
  - Actualización base de datos local

    - Proceso matado
- HIGH
    - Hash malicioso detectado
    - Archivo puesto en cuarentena (automática o manual)

9. Gestión de usuarios (PAM + Login)

El sistema deberá incluir autenticación de usuarios según los grupos a los que pertenezca dicho usuario.

Requisitos:

- Implementar login
- Usar PAM para autenticación
- Definir roles:
    - **admin_user** (administrador)
    - **common_user** (usuario común)

El programa debe ser capaz de verificar si un usuario pertenece a uno de estos grupos utilizando el nombre de usuario y contraseña, regresando un mensaje de éxito o fallo.

* Si un usuario pertenece a ambos grupos, se tomará como administrador.

Permisos

Acción	Administrador	Usuario
Ver métricas del sistema	✔	✔
Ver procesos activos	✔	✔
Ver alertas	✔	✔
Ver listado de archivos analizados	✔	✔

Activar y desactivar escaneo archivos (cargar y descargar eBPF)	✔	X
Activar y desactivar métricas sistema	✔	X
Actualizar base de datos local de hashes malignos	✔	X
Obtener información de proceso con su PID	✔	X
Matar un proceso utilizando su PID	✔	X
Configurar la ruta a monitorear	✔	X
Activar y desactivar escaneo de procesos activos	✔	X
Poner archivos en cuarentena	✔	X
Visualizar archivos en cuarentena	✔	X
Finalizar logwatch	✔	X

Restricción importante

- El control de permisos debe realizarse en el programa en C.
- Los datos procesados son enviados al frontend en formato JSON mediante HTTP o WebSockets.
- Se podrá agregar nuevos usuarios en tiempo de ejecución por medio de la consola.

Concurrencia (obligatorio)

El programa deberá implementar al menos 3 hilos:

Thread 1 → Monitoreo: monitoreo del sistema, listado de procesos y sistema de alertas.

Thread 2 → Sistema de archivos: consumo del Ring Buffer eBPF, cálculo de hashes, listado de archivos y cuarentena.

Thread 3 → Canal IPC: bloqueado en sys_ipc_channel_receive esperando notificaciones de log_watch (no puede compartirse con el hilo de monitoreo periódico porque la llamada bloquea).

Nota: Se pueden utilizar más hilos si se considera necesario (por ejemplo, para el servidor HTTP/WebSocket). Debe protegerse con mutex el acceso a estructuras compartidas (alertas, blacklist, métricas).

10. Integración con log_watch (canal IPC)

El daemon inicia sys_start_log_watch al arrancar con los parámetros de su archivo de configuración y detiene el monitoreo con sys_stop_log_watch al finalizar. Un hilo del daemon debe permanecer bloqueado en sys_ipc_channel_receive; al recibir una notificación, genera una alerta MEDIUM 'Palabra clave detectada en log' con el archivo y la línea reportados. Está prohibido el polling.

2.2.4 Dashboard Web (Frontend)

El sistema deberá incluir una interfaz gráfica accesible mediante navegador web, cuya función será visualizar en tiempo real la información generada por el programa en espacio de usuario (daemon en C), así como presentar alertas de seguridad y permitir la interacción básica del usuario según su rol.

1. Visualización de métricas

El dashboard deberá mostrar gráficas a su elección. Estas gráficas deben alimentarse con los datos generados por el daemon en tiempo real.

- Páginas activas vs. Páginas inactivas
- Uso de Memoria y Swap en el tiempo
- Memoria Libre vs. Memoria Caché vs. Memoria Utilizada
- Fallos de Página Menores (pgfault) vs. Mayores (pgmajfault)

A su vez deberá de mostrar un tabla con el top 20 procesos con mayor consumo en memoria.

El administrador tendrá la opción de activar y desactivar el escaneo de las métricas del sistema.

2. Panel de alertas

El dashboard deberá incluir una sección visible de alertas de seguridad generadas por el sistema.

Cada alerta deberá mostrar:

- tipo de evento (archivos, sistema, sesión, logs)
- descripción
- nivel de severidad (LOW, MEDIUM, HIGH)
- fecha y hora del evento

Las alertas deben mostrarse de forma ordenada (más recientes primero). Las alertas deberán diferenciarse visualmente según su severidad y un mecanismo visual (ej: color):

- LOW → informativo
- MEDIUM → advertencia
- HIGH → crítico

3. Visualización de archivos analizados

El dashboard deberá incluir una sección donde se muestre:

- lista de archivos analizados
- estado del archivo:
    - benigno
    - infectado
- timestamp de última modificación

El administrador tendrá la siguientes funcionalidades extra:

- Activar y desactivar el escaneo de archivos (adjuntar/attach y desadjuntar/detach el programa eBPF mediante libbpf).
- Agregar un archivo en cuarentena
- Visualizar los archivos en cuarentena

4. Visualización de amenazas detectadas

Cuando se detecte un hash malicioso, el dashboard deberá mostrar:

- Nombre de la amenaza (firma)
- Archivo afectado
- Descripción

El administrador tendrá la opción de actualizar la base de datos de hashes malignos subiendo un archivo en formato JSON (se debe validar que tenga el formato indicado).

Base de datos a utilizar: Proyecto

5. Visualización de procesos activos

El dashboard deberá mostrar un listado de los procesos con mayor consumo, donde cada proceso deberá mostrar:

- PID
- Nombre
- Porcentaje de uso de memoria

El administrador tendrá la siguientes funcionalidades extra:

- Obtener la información de procesos utilizando su PID.
- Activar y desactivar el escaneo de procesos activos.
- Matar cualquier proceso, para esto se debe de mostrar un mensaje de confirmación por motivos de estabilidad del sistema.

6. Análisis de palabras clave

El dashboard ahora deberá permitir la creación de nuevos logwatch, teniendo la posibilidad de ingresar la ruta central y las rutas a analizar por cada uno desde el frontend (queda a discreción del estudiante como solicitar estos datos).

A su vez se podrá visualizar el listado de logwatch activos (tabla), cada uno con su propio ID y la opción de terminarlo (finalizar el logwatch específico).

7. Sistema de autenticación

El dashboard deberá incluir un sistema de inicio de sesión que permite autenticarse contra el sistema (PAM).

El usuario deberá ingresar:

- nombre de usuario
- contraseña

Si el usuario no pertenece a ninguno de los grupos (*common_user*, *admin_user*) no deberá poder acceder al sistema.

* El dashboard deberá adaptar su comportamiento según el tipo de usuario autenticado y los permisos que tenga.

Consideraciones técnicas

- El dashboard debe desarrollarse con HTML, CSS y JavaScript.
- No es obligatorio el uso de frameworks, es opcional.
- Debe consumir los datos generados por el programa en C (daemon).

2.3 Requerimientos técnicos

1. El sistema deberá desarrollarse sobre Linux, trabajando directamente con el kernel.
2. Las syscalls deberán implementarse en C e integrarse correctamente en el kernel.
3. El kernel utilizado debe ser la versión 6.12.69 LTS de Linux.
4. El daemon deberá desarrollarse en C y ejecutarse de forma continua.
    a. El daemon deberá implementar concurrencia con al menos tres hilos: monitoreo del sistema, escaneo de archivos (eBPF) y canal IPC.
5. La comunicación entre el daemon y el dashboard deberá realizarse mediante HTTP o WebSockets, utilizando formato JSON.
6. El sistema deberá implementar autenticación mediante **PAM**.
7. El control de roles y permisos deberá gestionarse en el daemon.
8. El sistema deberá utilizar el archivo `blacklist.json` como base de firmas.
9. El cálculo de hashes deberá realizarse con el algoritmo criptográfico SHA-256.
10. El sistema deberá manejar errores sin finalizar abruptamente.
11. El dashboard deberá desarrollarse con tecnologías web (HTML, CSS, JavaScript).
12. Se permite el uso de frameworks (siempre que se documente el porqué de su uso.
13. No se permite lógica de detección, clasificación de severidad ni filtrado por listas de hashes o firmas en el kernel. Excepciones: el filtro por ruta del sensor eBPF y la coincidencia de palabra clave de sys_start_log_watch (Práctica 3).
14. La intercepción de eventos de archivos debe realizarse mediante código eBPF compilado con Clang/LLVM. El daemon en C deberá utilizar la librería ***libbpf*** para cargar el programa en el kernel y consumir los eventos desde el Ring Buffer
15. El kernel compilado debe incluir soporte para eBPF: CONFIG_BPF=y, CONFIG_BPF_SYSCALL=y, CONFIG_BPF_EVENTS=y, CONFIG_DEBUG_INFO_BTF=y y CONFIG_FTRACE_SYSCALLS=y.

3. Entregables

Tipo	Descripción
Repositorio GitLab	Link del repositorio y rama donde se subió el código del kernel modificado, las syscalls implementadas, el programa intermedio y el dashboard web. Todo el código realizado para el proyecto debe de ir dentro de la carpeta **ProyectoUnico_2S2026.**
Modificación llamadas al kernel	Código modificado del kernel 6.12.69 con las syscalls de las Prácticas 2, 3 y 4 integradas. Debe seguir la estructura original del kernel e incluir únicamente los archivos modificados. Todo debe estar dentro de la carpeta Kernel.
Programa intermedio	Código en C con ejecución en segundo plano (daemon) que invoque las syscalls, procese la información, realice el análisis (detección, alertas, severidad) y envíe los datos al frontend en formato JSON mediante HTTP o WebSockets. Debe implementar concurrencia (mínimo 2 hilos) e incluir el ejecutable compilado. Todo dentro de la carpeta **ProgramaIntermedio**. Incluye el código fuente del sensor eBPF (.bpf.c) y su Makefile, dentro de la subcarpeta ProgramaIntermedio/ebpf.
Dashboard web (frontend)	Código de interfaz gráfica que permite visualizar el estado del sistema, visualizar gráficas, visualizar alertas y escaneo de archivos. Todo dentro de la carpeta **Dashboard.**
Manual Técnico	Documento Markdown que explique la integración de las syscalls reutilizadas, el sensor eBPF (mapas, Ring Buffer y comunicación con el daemon), la funcionalidad del programa intermedio (manejo de errores e hilos) y su conexión con el frontend.
Manual de Usuario	Documento Markdown con capturas e instrucciones del uso del dashboard web (frontend) tanto para administrador como usuario.

4. Material de apoyo

- Documentación oficial de eBPF y libbpf: https://docs.kernel.org/bpf/
- Documentación oficial del kernel de Linux: https://docs.kernel.org
- Manual man proc, man bpf, man 2 syscall.

- Documentación de Linux-PAM: https://www.linux-pam.org/Linux-PAM-html/
- Documentación de OpenSSL (EVP_Digest / SHA-256): https://docs.openssl.org/

5. Recursos y herramientas a utilizar

Software:

- GNU/Linux (distribución basada en Debian).
- Clang / LLVM (para la compilación del programa eBPF).
- libbpf y bpftool (para carga e interacción con eBPF desde espacio de usuario).
- GCC, Make.
- Librería PAM (libpam0g-dev) para autenticación.
- Librería OpenSSL (libssl-dev) para el cálculo de hashes SHA-256.
- Git (control de versiones).
- Editor de código: VS Code o Vim.

Lecturas recomendadas:

1. Capítulos sobre llamadas al sistema y espacio de usuario/kernel de *Understanding the Linux Kernel*.
2. Guías sobre `/proc` y herramientas de monitoreo (`top`, `htop`, `ps`).

6. Cronograma

El cronograma describe las etapas clave del proyecto, los plazos estimados para cada una, y el proceso de asignación, elaboración y calificación de las tareas. Los estudiantes deberán seguir este plan para asegurar que el proyecto avance de manera organizada y cumpla con los plazos establecidos. Cada fase incluye la asignación de tareas, el tiempo estimado para su elaboración, y el momento de su calificación.

Tipo	Fecha Inicio	Fecha Fin
Asignación de Proyecto	09/10/2026	09/10/2026
Elaboración	09/10/2026	27/10/2026
Calificación	28/10/2026	30/10/2026

7. Rúbrica de Calificación

7.1 Requisitos para optar a la calificación

Antes de la evaluación del proyecto, los estudiantes deben cumplir con los requisitos que se indiquen en esta sección.

Tema	Descripción	Cumple (Sí/No)
Cumplimiento de la tecnología establecida	Programa intermedio realizado en el lenguaje C.	
Uso de herramientas requeridas	Uso del kernel Linux en la versión LTS 6.12.69 con soporte eBPF habilitado y la integración de las syscalls implementadas en la Práctica 4.	
Gestión y entregas de práctica	El repositorio debe contener el código modificado del kernel, el sensor eBPF, el programa intermedio (daemon) y el dashboard web. Todo organizado dentro de la carpeta llamada **ProyectoUnico_2S2026**.	
Documentación obligatoria	Se deben entregar los manuales Técnico y de Usuario en formato Markdown.	
Manejo de hilos.	El programa intermedio (daemon) debe utilizar concurrencia con al menos 3 hilos (monitoreo del sistema, escaneo de archivos eBPF y canal IPC).	
Entrega en Uedi/Classroom	Se debe haber realizado la entrega de la rama del repositorio en la plataforma correspondiente a su tutor.	

7.2 Resumen de Puntuaciones

Área	Puntos Totales	Puntos Obtenidos
HABILIDADES		
Implementación de Syscalls (kernel) y Sensor eBPF.	40	
Programa en C (daemon: monitoreo, escaneo, alertas, concurrencia, cuarentena)	25	
Dashboard Web (visualización, alertas, control de escaneo)	10	
Gestión de Usuarios (PAM + Permisos)	5	
Cambio/Funcionalidad Adicional (Durante la calificación)	10	
Subtotal HABILIDADES	90	
CONOCIMIENTO		
Manual Técnico	5	
Manual de Usuario	5	
Subtotal CONOCIMIENTO	10	
TOTAL GENERAL	**100**	

7.3 Comentarios Generales

7.3 Comentarios Generales..... 18

_____

_____

_____

_____

No.	Criterio de Evaluación	Punteo Máximo	Satisfactorio (100–61%)	Necesita Mejorar (60–0%)	Punteo Obtenido
Área de Habilidades					
1	Capa Kernel (Syscalls + eBPF)	40	Las syscalls reutilizadas (process_info, processes_state, canal IPC y log_watch) están integradas en el kernel 6.12.69, devuelven datos correctos y no afectan la estabilidad. El programa eBPF intercepta eventos del sistema de archivos de forma segura, ignora los eventos del propio daemon y transmite los metadatos al espacio de usuario mediante Ring/Perf Buffer usando libbpf.	Las syscalls devuelven datos incorrectos, no están integradas o causan Kernel Panic. El programa eBPF no intercepta eventos, no logra adjuntarse al kernel, no transmite datos por el Ring Buffer o genera un bucle de eventos por no ignorar al daemon.	
			25 - 40 pts	0 - 24 pts	
2	Daemon en C (Concurrencia y Análisis JSON)	25	El daemon ejecuta al menos 2 hilos en segundo plano, consume eventos de eBPF mediante libbpf, lee /proc/meminfo y /proc/vmstat, calcula SHA-256 con OpenSSL, contrasta con blacklist.json, gestiona la cuarentena de archivos e impide terminar PIDs protegidos. Recibe las notificaciones de log_watch con sys_ipc_channel_receive de forma bloqueante, sin polling.	No implementa la concurrencia requerida, falla la lectura de métricas o el cálculo de hashes, realiza filtrados dentro del kernel o el daemon se detiene de forma inesperada, o usa polling en lugar de bloquearse en el canal IPC.	
			16 - 25 pts	0 - 15 pt	
3	Dashboard Web (Frontend)	10	Presenta gráficas en tiempo real de memoria, swap y fallos de página. La tabla del Top 20 de procesos y el panel de alertas se actualizan correctamente diferenciándose por color. Oculta opciones según el rol del usuario.	Interfaz estática, no recibe datos del Daemon, o no diferencia visualmente el nivel de severidad de las alertas.	
			7 - 10 pts	0 - 6 pts	
4	Gestión de usuarios (PAM + Permisos)	5	Autenticación real usando PAM en C. Aplica reglas estrictas de acceso (admin_user vs common_user) rechazando peticiones no autorizadas desde el backend.	El login está simulado (quemado en código), PAM no funciona, o las restricciones de seguridad solo se aplican ocultando botones en el frontend.	
			4 - 5 pts	0 - 3 pts	

5	Cambio / Funcionalidad Adicional	10	Implementa una mejora relevante solicitada durante la revisión, integrándose al flujo de trabajo actual sin romper el sistema.	El cambio requerido no funciona, está incompleto o causa inestabilidad en el proyecto base.	
			7 - 10 pts	0 - 6 pts	
Sub-total		90			
Área de Conocimiento					
6	Redacción del Manual Técnico	5	Explica a detalle la arquitectura concurrente, las syscalls reutilizadas y su integración y cómo se comunica el programa eBPF con el Daemon a través de mapas/buffers compartidos .	Documentación pobre, carece de explicaciones técnicas sobre la implementación del código o falta explicación de la capa kernel.	
			4 - 5 pts	0 - 3 pts	
7	Elaboración del Manual de Usuario	5	Guía clara con capturas de pantalla, explicando los pasos para encender el Daemon, hacer login y entender las gráficas del dashboard.	Manual incompleto, sin imágenes o que no explica las diferencias de roles de usuario en el sistema.	
			4 - 5 pts	0 - 3 pts	
Sub-total		15			
Total		100			

Comentarios Generales

Apartado para comentar si hubo algún inconveniente durante la calificación o comentario.

_____
_____
_____
_____
_____